

ScholarLink

An LLM-Powered Platform for Research Discovery, Topic Extraction, and Academic Collaboration

Project Proposal for UCS503

Hemant Chugh*

Diljeet Kaur[†]

Saurabh Shivhare[‡]

Divyam[§]

Branch: Artificial Intelligence & Machine Learning
Thapar Institute of Engineering and Technology (TIET), Patiala
Course: UCS503 — Software Engineering
Submission Date: 18 August 2026

GitHub: <https://github.com/simplernix/ScholarLink-ucs503.git>

1 Introduction

Research and project work produced within a department is scattered across personal drives, email threads, and printed submissions, with no shared, searchable record of who is working on what. A junior student looking for a mentor, or a student wanting to build on a senior's project, has no reliable way to discover relevant work beyond informal channels. This project proposes ScholarLink, a web platform where professors and students publish research papers and project reports, tag co-authors, and build a public academic profile. A large language model (LLM) reads each uploaded paper to automatically extract the topics/domains it belongs to and generate a concise summary of the future scope of work identified in it, so that other students can discover relevant work by topic and contact the original authors to collaborate or seek guidance.

2 Problem Statement

Departments lack a lightweight, searchable system that connects published student/faculty research to the people best placed to extend it. Existing repositories (departmental drives, printed archives, or general platforms like ResearchGate) either require heavy manual curation, are aimed at large-scale use outside academia, or provide no structured way to identify a paper's future scope and no built-in mechanism for a junior student to request mentorship from a specific author. There is a need for a system where uploading a paper automatically classifies it by topic, surfaces what work remains open, and lets interested students contact the right person directly from the paper's page.

*Roll No: 1024240042

[†]Roll No: 1024240058

[‡]Roll No: 1024240035

[§]Roll No: 1024240061

3 Objectives

- Build a role-based web application (Student, Professor, Admin) with secure JSON Web Token (JWT) authentication.
- Allow professors and students to upload research papers/project reports and tag co-authoring students at upload time.
- Integrate an LLM pipeline that extracts text from the uploaded PDF and returns structured topic tags and a future-scope summary.
- Provide topic-based search and browse so students can discover papers relevant to their interests.
- Build a public profile page for every user summarising their past papers/projects, so junior students can identify potential mentors.
- Implement a collaboration-request workflow so a student can contact an author directly through the platform, with in-app notifications.
- Follow test-driven development (TDD) for the topic-extraction and collaboration-request logic, and automate build-test-deploy with Continuous Integration/Continuous Deployment (CI/CD).

4 Scope of the Project

The system covers the complete lifecycle of a paper within a single institution: user authentication and role management, paper upload with co-author tagging, automated LLM-based topic and future-scope extraction, topic-based search, author profiles, and a collaboration-request workflow with notifications. Scope is limited to a single institution for the first release; cross-institution federation, plagiarism/duplicate-submission detection, and full in-app real-time chat are identified as future extensions and are out of scope for this proposal.

4.1 Scope Refinement

To keep the project deliverable within a single semester without losing academic depth, the initial feature list was narrowed as follows. Collaboration is implemented as a request/accept workflow with a single follow-up message, not a full real-time chat system — this avoids building a WebSocket-based messaging service that adds implementation effort without adding to the core learning outcomes of the project. A citation-network graph view, showing which papers cite which others, was considered and dropped entirely, since the project already demonstrates NLP-driven insight through topic extraction and future-scope summarisation, and a third analytical feature would not add proportional value. Semantic (embedding-based) search is retained but reclassified as an optional extended feature, to be built only once the core upload, LLM-extraction, and keyword/tag search are complete and stable — if the project falls behind schedule, this is the first feature to be cut. Topic and future-scope extraction is kept as the core, always-in-scope LLM feature, implemented as a single well-prompted API call per paper rather than a fine-tuned model. This narrower scope keeps the system architecture as a modular monolith (Route → Service → Repository) rather than true microservices, and keeps notifications in-app rather than adding external message-broker infrastructure such as Kafka or RabbitMQ.

5 Existing Systems and Their Limitations

Platforms such as ResearchGate and Google Scholar index published research at a global scale but are not designed for an internal departmental setting — they offer no tagging of student co-authors, no institution-specific mentorship workflow, and no automatic summary of a paper’s open future work. Institutional repositories typically store PDFs with manual metadata entry and no topic intelligence, so discovery depends entirely on knowing what to search for. Few, if any, of these tools surface the remaining open work from a paper, which is precisely the information a prospective collaborator needs first. ScholarLink targets that gap: automatic topic tagging and future-scope extraction at a scope built specifically for intra-institution discovery and mentorship, rather than global publication indexing.

6 Proposed System

The proposed system is a three-tier web application. A React frontend provides role-specific screens (upload, browse/search, paper detail, profile, requests). A FastAPI backend exposes REST endpoints and enforces business rules in a Route → Service → Repository layering, keeping business logic out of route handlers so the layers stay orthogonal to one another — for example, swapping the LLM provider later would only require changes within the LLM service layer. PostgreSQL is the system of record, with SQLAlchemy and Alembic handling ORM mapping and schema migrations. On upload, the backend extracts text from the PDF and enqueues an asynchronous job that calls an LLM API to generate topic tags and a future-scope summary, so uploads are not blocked on API latency; results are written back to the paper’s record once ready and the paper becomes searchable.

6.1 Core Entities

The domain model centers on seven entities: User, Paper, PaperAuthor, Topic, PaperTopic, PaperInsight, CollaborationRequest, and Notification. A Paper has one or more PaperAuthor links, each pointing to a User with a role of Owner (the uploading professor/student) or Tagged Student, which is how co-authorship is represented without requiring every collaborator to have uploaded the paper themselves. A Paper is linked to one or more Topics through PaperTopic, which also stores a relevance score returned by the LLM. Each Paper has exactly one PaperInsight record holding the generated summary and future-scope text, which can be regenerated if the paper is updated. A CollaborationRequest links a sending User to a receiving User in the context of a specific Paper, and its status changes trigger a Notification.

7 Feature List (Module-Wise)

7.1 Authentication and User Management

- Login/logout with JWT-based sessions and hashed passwords.
- Role-based access control: Student, Professor, Admin.
- Admin-only user management — verify institutional email domain, deactivate accounts.

7.2 Paper and Project Upload

- Upload a research paper or project report (PDF) with title, abstract, and tagged co-author students.
- Version field for updated/revised uploads of the same work.

- Paper status lifecycle: Processing → Published, reflecting the async LLM extraction step.

7.3 LLM-Based Topic and Future-Scope Extraction

- Text extraction from the uploaded PDF (PyMuPDF).
- A single structured LLM call returns: a list of topic tags, a short summary, and a future-scope-of-work paragraph.
- Extracted topics are matched against an existing Topic table where possible, and new topics are created otherwise, to avoid unbounded tag sprawl.

7.4 Search and Discovery

- Browse and filter papers by topic tag, author, or institution department.
- Keyword search over title, abstract, and generated summary.
- Semantic (embedding-based) search as an optional extended feature.

7.5 Profiles and Portfolio

- Public profile listing a user's papers/projects, role in each (owner or tagged student), and topic areas of interest.
- Editable bio and areas of expertise, so junior students can identify who to approach for guidance.

7.6 Collaboration Requests

- A student can send a collaboration/mentorship request to a paper's author with a short message.
- Author can accept or decline; an accepted request reveals contact details for follow-up outside the platform.

7.7 Notifications

- Event-driven alerts: new collaboration request, request accepted/declined, paper-processing complete.
- In-app notifications first; no external message broker required at this stage.

7.8 Testing and DevOps

- Unit tests (pytest) for topic-matching, request-status transitions, and access-control logic, following a red → green → refactor TDD cycle.
- Black-box tests for flows such as login and paper upload, and integration tests across upload → extraction → search.
- GitHub Actions pipeline: test → build → deploy on every push.

8 System Workflow — Life of a Paper Upload

1. Professor logs in; a JWT is issued with role = Professor.
2. Professor opens the upload screen, attaches the PDF, tags co-authoring students, and submits; the frontend calls `POST /papers`.
3. The FastAPI route delegates to the service layer, which stores the file, creates the Paper and Paper-Author records, and sets status = Processing.
4. A background worker extracts text from the PDF and calls the LLM API with a structured prompt requesting topics, a summary, and the future scope of work.
5. The service layer matches returned topics against the Topic table (creating new ones where needed), writes the PaperInsight record, and sets status = Published.
6. Tagged students and the uploading professor receive a notification that processing is complete.
7. A browsing student searches by topic, opens the paper, reads the LLM-generated future-scope summary, and sends a CollaborationRequest to the author.
8. The author receives a notification, reviews the request, and accepts or declines it; on acceptance, both parties can see each other's contact details.

9 Technology Stack

As this is a proposal rather than a final implementation report, Table 1 lists the technology proposed for each layer alongside credible alternatives that were considered. The proposed choice reflects the best fit for a single-semester, single-institution deployment; the alternatives remain open for revision during design review if the team's familiarity, deployment environment, or institutional constraints change.

10 Expected Outcome

At the end of this project, the deliverable will be a working, tested ScholarLink platform covering authentication, paper upload with co-author tagging, automated LLM-based topic and future-scope extraction, topic search, public author profiles, and a collaboration-request workflow with notifications. The system will be containerised and deployed via an automated CI/CD pipeline, with unit and integration test coverage over the topic-extraction and collaboration-request logic.

11 Limitations

While ScholarLink addresses a real discovery and mentorship gap within a department, the proposed scope carries a number of known limitations that should be considered alongside the expected outcomes described above.

- **Single-institution scope.** The system is designed and evaluated for one institution's user base; cross-institution federation, shared topic taxonomies across departments, and inter-university discovery are not part of this release.

Layer	Proposed Technology	Alternatives Considered
Frontend	React (Hooks, Router, fetch/axios)	Vue.js; Angular; Next.js
Backend	FastAPI + Pydantic (validation), JWT authentication	Django REST Framework; Flask; Node.js with Express
ORM / Migrations	SQLAlchemy + Alembic	Django ORM; Prisma; raw SQL with a lightweight query builder
Database	PostgreSQL	MySQL / MariaDB; MongoDB (if a document-oriented model were preferred)
File Storage	S3-compatible object storage for uploaded PDFs	Local filesystem storage (for a smaller pilot); Google Cloud Storage; Azure Blob Storage
LLM Layer	Claude/OpenAI API for topic extraction and future-scope summarisation, via an async worker queue	Google Gemini API; a self-hosted open-source model (e.g. Llama-family) for cost/privacy control
Search	PostgreSQL full-text search; pgvector-based semantic search as an optional extended feature	Elasticsearch / OpenSearch; Meilisearch; Typesense
Testing	pytest, TDD, black-box and integration tests	unittest; Postman/Newman for API-level black-box tests
CI/CD	GitHub Actions → Docker / Docker Compose → deploy	GitLab CI/CD; Jenkins; CircleCI
Version Control	Git/GitHub, feature-branch workflow (feature/auth, feature/upload, feature/llm-extraction, etc.)	GitLab; Bitbucket

Table 1: Proposed technology stack and alternatives considered.

- **Dependence on external LLM APIs.** Topic tagging and future-scope extraction rely on a third-party LLM provider, so extraction quality, latency, and ongoing cost are outside the project’s direct control, and an outage or rate limit on the provider side would pause new-paper processing.
- **No plagiarism or duplicate-submission detection.** The system does not verify originality or check whether an uploaded paper duplicates existing work, so content integrity ultimately relies on the uploader and institutional policy rather than automated checks.
- **Text extraction limits.** PDF text extraction is unreliable for scanned or image-only documents and for papers with complex multi-column layouts, figures, or non-English text, which can degrade the quality of the generated topics and summary.
- **No real-time collaboration.** Collaboration is limited to a request/accept workflow with a single follow-up message rather than in-platform chat, so extended discussion between collaborators still has to move to email or another channel.
- **Semantic search is an optional extended feature.** Discovery in the base release depends on keyword and tag search; embedding-based semantic search is only delivered if time permits, so some conceptually related papers may not surface unless they share explicit tags or keywords.
- **Manual verification of AI output.** Generated topics and future-scope summaries are not fact-

checked against the paper’s actual content beyond the single LLM call, so occasional mis-tagged topics or inaccurate summaries are possible and would need author correction.

- **Scalability of the modular monolith.** The Route → Service → Repository monolith is appropriate for a single-department deployment but was not designed for very large, multi-institution traffic; scaling beyond that would require architectural changes such as splitting services or introducing a message broker.

12 Conclusion

This proposal outlines a practical, end-to-end solution to a real problem faced within academic departments — research and project work that exists but is undiscoverable, and no structured way for junior students to find the right mentor. By layering the system cleanly (Route → Service → Repository), using the LLM for a single well-scoped extraction task rather than an open-ended assistant, and building collaboration directly into the paper-discovery flow, the project aims to deliver both an academically rigorous software-engineering exercise and a genuinely usable tool.

13 Overview / Summary

ScholarLink is a proposed web platform that gives a department a shared, searchable record of the research and project work happening within it. Professors and students upload papers and project reports, tag co-authors, and build public academic profiles; an LLM automatically extracts topic tags and a future-scope-of-work summary from each upload, so other students can discover relevant work by topic rather than by word of mouth. A built-in collaboration-request workflow then lets an interested student contact the original authors directly, with in-app notifications guiding the exchange from first message to accepted contact.

Architecturally, the system is a three-tier application — a React frontend, a FastAPI backend organised as Route → Service → Repository, and PostgreSQL for storage — kept deliberately as a modular monolith for a single-semester, single-institution scope. Core features include role-based authentication, paper upload with co-author tagging and an async LLM-extraction pipeline, topic-based search, author profiles, and the collaboration-request/notification workflow, all built with test-driven development and a GitHub Actions CI/CD pipeline. Known limitations include reliance on a third-party LLM provider, the absence of plagiarism detection and real-time chat, and unverified AI-generated summaries — trade-offs made deliberately to keep the project achievable within one semester while still delivering a genuinely useful tool for intra-institution research discovery and mentorship.